

Chapter 13: Systems Programming (Process Management)

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes
from Raj Sunderraman, Michael Weeks, and Yuan Long

lseek() Example

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main(void) {
    int fd;
    char buffer[20];
    ssize_t bytesRead;

    fd = open("example.txt", O_RDONLY); // Open the file in read-only mode
    if (fd == -1) {
        perror("open");
        return 1;
    }

    // Move the file offset to byte 5 (from the beginning)
    if (lseek(fd, 5, SEEK_SET) == -1) {
        perror("lseek");
        close(fd);
        return 1;
    }

    bytesRead = read(fd, buffer, sizeof(buffer) - 1); // Read from the new file offset
    buffer[bytesRead] = '\0';
    printf("Data read: %s\n", buffer);
    close(fd);
    return 0;
}
```

unlink()

- unlink() is a Unix/POSIX system call used to remove a name (link) from the filesystem.
- If the file has no other hard links and no process has it open, the file's data is deleted immediately.
- If the file is open by a process, the data remains until the last file descriptor is closed.

unlink() Example

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
int main(void) {
    int fd;
    char buffer[50];
    // Create and open a file
    fd = open("unlink_example.txt", O_CREAT | O_WRONLY, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    write(fd, "Hello..unlinking example\n", 25);
    sleep(10); // Now check the file..it exists
    // Remove the filename from the directory
    if (unlink("unlink_example.txt") == -1) { //Now the filename does not exist
        perror("unlink");
        close(fd);
        return 1;
    }
    printf("File unlinked, but still open.\n");
    // The file still exists until fd is closed
    sleep(10);
    close(fd);
    printf("File descriptor closed. File is now fully removed.\n");
    return 0;
}
```

Hard Link/Soft Link

- A hard link is another directory entry that points to the same inode as the original file.
 - Multiple filenames → same inode
 - All hard links are equal (no “original” file)
 - File data exists as long as at least one hard link remains
- A soft link (symbolic link) is a separate file that stores the pathname of another file. (works for both file and directory)
 - Points to a filename, not an inode
 - Has its own inode
 - Can point to a file across filesystems
 - Can become dangling if the target is deleted

Writing binary files

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE *wf;
    int number = 33;
    // Open the file in binary write mode
    wf = fopen("data.bin", "wb");
    if (wf == NULL) {
        perror("Error opening file");
        return 1;
    }
    // Write the integer to the file
    fwrite(&number, sizeof(int), 1, wf);
    fclose(wf);
    return 0;
}
```

Check the file
content after writing

Reading binary files

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE *rf;
    int num;
    rf = fopen("data.bin", "rb"); // Open the file in binary read mode
    if (rf == NULL) {
        perror("Error opening file");
        return 1;
    }
    if (fread(&num, sizeof(int), 1, rf) != 1) { //read the integer
        perror("Error reading file");
        fclose(rf);
        return 1;
    }
    fclose(rf);
    printf("The number read from the file is: %d\n", num);
    return 0;
}
```

Process

- A **process** is an instance of a running program.
 - Not the same as “program” or “processor”
- Process provides each program with two key abstractions:
 - Logical control flow
 - ✦ Each program seems to have exclusive use of the CPU.
 - Private address space
 - ✦ Each program seems to have exclusive use of main memory.
- How are these Illusions maintained?
 - Process executions interleaved (multitasking)
 - Address spaces managed by virtual memory system

Process Management

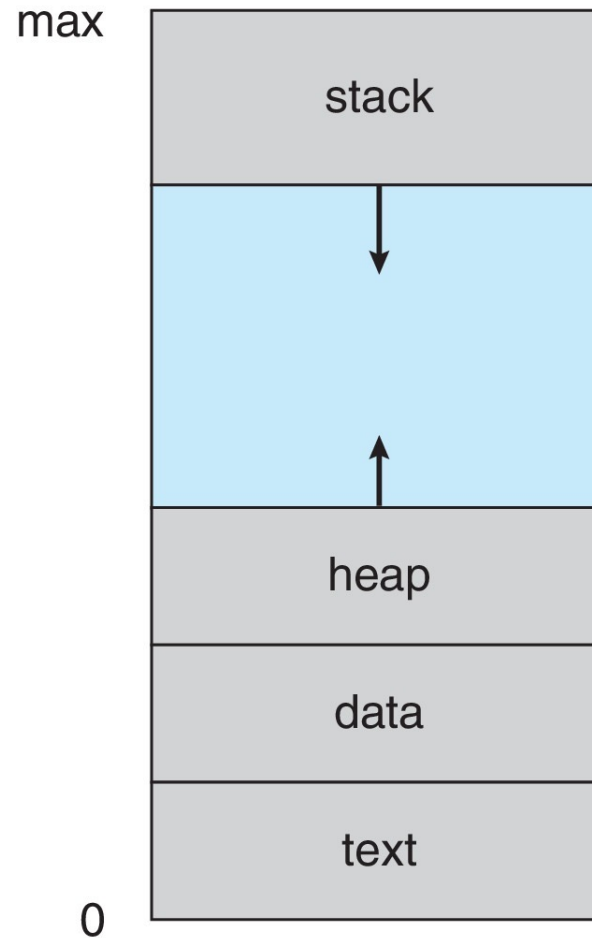
- Every process in UNIX has
 - some code
 - some data
 - a stack
 - a unique process ID (PID)

Init Process

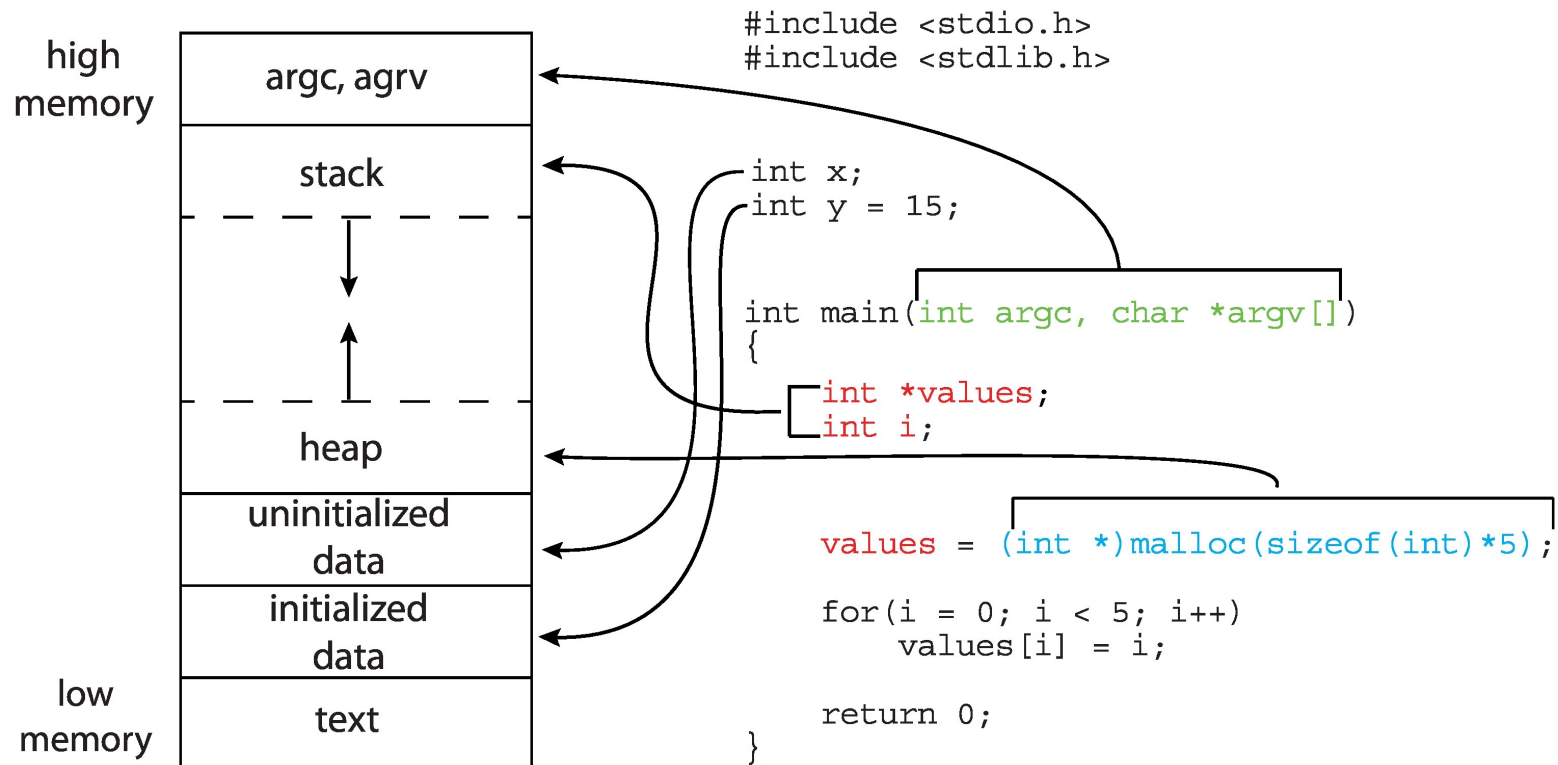
- When UNIX starts (boots), there is only one process, called *init*, with process ID = 1.
 - The only way to create a new process is to **duplicate an existing process**
 - So *init* is the ancestor of all subsequent processes.
- Initially, *init* duplicates (***forks***) several times and each child process replaces its code (***execs***) with the code of the executable ***getty*** which is responsible for user logins.

Process in Memory

The blank (blue) space is **available memory** for the heap and stack.



Memory Layout of a C Program



global data section is divided into (a) initialized data and (b) uninitialized data. A separate section is provided for the argc and argv parameters passed